

ELEG 55903: Programming for Power Electronics

Homework #6

Tyler Cash

ELEG 4593

HW #6

2 March 2026

Abstract:

In this homework assignment we will create a GCD implementation in VHDL. The GCD algorithm is as follows: continually subtract the smaller of A or B from the larger, storing the result in the larger, until both are the same, the resulting value is the GCD.

Table of Contents:

Abstract:.....	2
List of Figures:	3
Overview:.....	4
Modules:	4
GCD:	4
Testbench:.....	5
Conclusion:.....	5
Appendix A (Module Code):	7
Appendix B (GCD Code):.....	11
Appendix C (GCD Testbench):	17

List of Figures:

FIGURE 1. GCD ASM AND DATAPATH	6
FIGURE 2. GCD WAVEFORM 1	6
FIGURE 3. GCD WAVEFORM 2	6

Overview:

This project involves implementing a GCD in VHDL with the following inputs and outputs:

Inputs: clk, resetn, Xdat, Ydat, Yrst, A (8-bit), B (8-bit)

Outputs: Y (8-bit), Ydat, Xrst

The GCD algorithm continually subtracts the smaller of A or B from the larger, storing the result in the larger, until both are the same. The Algorithmic State Machine and Datapath are found in Figure 1. The requirements for this homework were to submit our code (found in the appendix), as well as a testbench to simulate the GCD, making sure to include the values of A = 25 and B = 30.

Modules:

For this homework assignment, and for future assignments, we began by creating a file to implement generic components needed for this project, as well as for future projects that could need those files, like a header file or library. For this homework, the components that were required were a 2x8mux, an 8-bit register, a comparator, a conditional inverse, and a ripple carry adder.

The GCD will use two 2x8muxs, for the A and B inputs, the selector switch will determine whether the input to the A and B registers will be from the direct A and B input, or the sum from the previous iteration. The next component is the 8-bit register, to hold the values, and intermediate values, as the GCD calculations are made. Then we use a comparator, to compare the values of A and B, to determine if one is greater, or if they are equal. Next we have the conditional inverse, we use this to do subtraction, by inverting the smaller number and passing that value to the ripple carry adder. The Ripple Carry Adder takes the larger of the two numbers, and the inverse of the smaller number, and then adds them together with a 1 carry bit, which serves to subtract the two numbers.

GCD:

Once we have the individual modules created, we move on to implementing the GCD. We instantiate the module and instantiate the components that we built in the modules file, mapping the correct inputs and outputs to the corresponding modules.

Once the modules and inputs / outputs are defined and assigned, we initiate the state logic for the GCD. The GCD only requires two states, S0, waiting for valid input data, and then S1, where the calculations are done.

Testbench:

The other requirement for this homework was to implement a testbench to simulate the design. One of the keys for a good test bench is to test different possibilities to ensure proper functionality of the design. For the GCD, the key things to check are for the behavior of the design when $A > B$, $A < B$, and when $A = B$. These are all expected possible inputs, so we should ensure that the combinations are all tested. For our testbench, we used $A = 25$ and $B = 30$ (required test), $A = 100$ and $B = 10$, $A = 9$ and $B = 9$, $A = 15$ and $B = 20$. With these inputs, we will have tested the different possibilities of A and B , and we can ensure the design behaves as expected, as you can see in Figure 2 and Figure 3.

Conclusion:

In conclusion, we have properly implemented the GCD design in VHDL, as well as an appropriate test bench, demonstrating the successful implementation of the GCD.

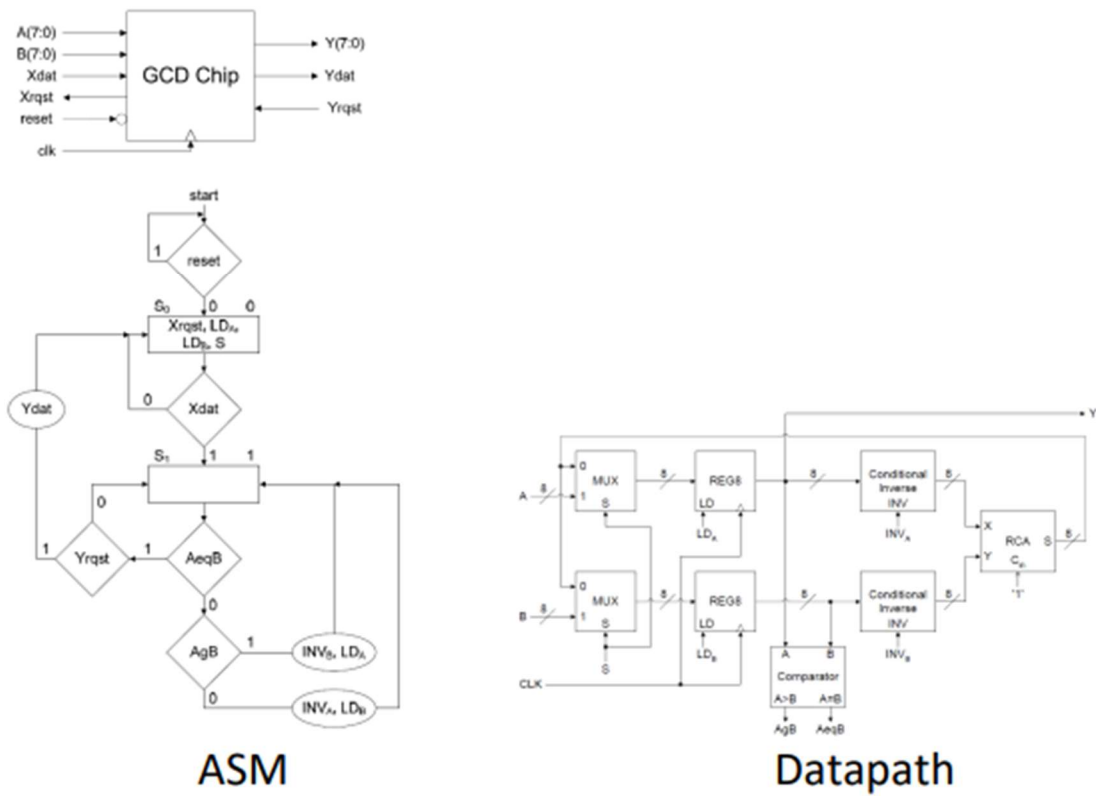


Figure 1. GCD ASM and Datapath

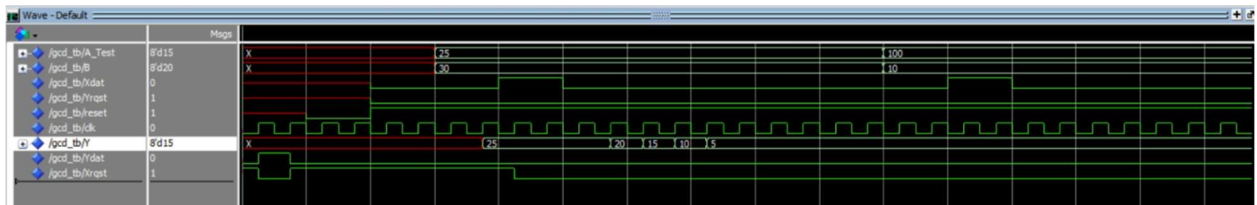


Figure 2. GCD Waveform 1

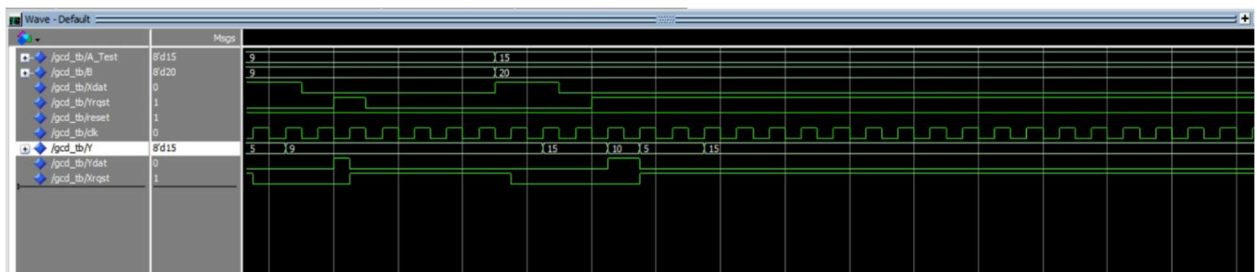


Figure 3. GCD Waveform 2

Appendix A (Module Code):

```
-----
-- Company: University of Arkansas (RIOT)
-- Engineer: Tyler Cash
--
-- Create Date: 27Feb2026
-- Last Updated: 27Feb2026
-- Design Name: GCD
-- Module Name: GCD TOP
-- Project Name: GCD
-- Target Devices: LCMXO3D-9400HC-5BG256C (MachXO3D_BreakoutBrd)
-- Tool versions: Lattice Diamond_x64 Build 3.12.0.240.2
-- Description:
-- Top module for implementing a GCD.
--
---- PinOut:
-- Signal      CPLD_Pin Description
--Push Buttons
-- Clk          J2      Clk_In (Input) [Onboard btn[0]]
-- PB_1         J5      PB1   (Input) [Onboard btn[1]]
-- PB_2         H2      PB2       (Input) [Onboard btn[2]]
-- PB_3         J1      PB3       (Input) [Onboard btn[3]]
--
-- Switches
-- X(0)         V2      SW0       (Input) [Onboard sw[0]]
-- X(1)         U2      SW1       (Input) [Onboard sw[1]]
-- X(2)         U1      SW2       (Input) [Onboard sw[2]]
-- X(3)         T2      SW3       (Input) [Onboard sw[3]]
-- X(4)         T1      SW4       (Input) [Onboard sw[4]]
-- X(5)         R2      SW5       (Input) [Onboard sw[5]]
-- X(6)         R1      SW6       (Input) [Onboard sw[6]]
-- X(7)         P2      SW7       (Input) [Onboard sw[7]]
-- resetP1      SW8       (Input) [Onboard sw[8]]
-- Xdat         N2      SW9       (Input) [Onboard sw[9]]
-- YrqstV2      SW10 (Input) [Onboard sw[10]]
-- Sw_11        P1      SW11 (Input) [Onboard sw[11]]
-- SW_12        P1      SW12 (Input) [Onboard sw[12]]
-- SW_13        P1      SW13 (Input) [Onboard sw[13]]
-- SW_14        P1      SW14 (Input) [Onboard sw[14]]
-- SW_15        P1      SW15 (Input) [Onboard sw[15]]
--
-- LEDs
```

```

-- Y(0)      G1    LED0   (Output) [Onboard led[0]]
-- Y(1)      G2    LED1   (Output) [Onboard led[1]]
-- Y(2)      F1    LED2   (Output) [Onboard led[2]]
-- Y(3)      F2    LED3   (Output) [Onboard led[3]]
-- Y(4)      E1    LED4   (Output) [Onboard led[4]]
-- Y(5)      E2    LED5   (Output) [Onboard led[5]]
-- Y(6)      E3    LED6   (Output) [Onboard led[6]]
-- Y(7)      E5    LED7   (Output) [Onboard led[7]]
-- Ydat      E6    LED8   (Output) [Onboard led[8]]
-- XrqstC3   LED9   (Output) [Onboard led[9]]
-- LED_10    B2    LED10   (Output) [Onboard led[10]]
-- LED_11    A2    LED11   (Output) [Onboard led[11]]
-- LED_12    B3    LED12   (Output) [Onboard led[12]]
-- LED_13    A3    LED13   (Output) [Onboard led[13]]
-- LED_14    B4    LED14   (Output) [Onboard led[14]]
-- LED_15    A4    LED15   (Output) [Onboard led[15]]

```

```

--
-- Revisions:--

```

```

--
-- Revision 0.01 -

```

```

-- File Created;

```

```

--
-- Additional Comments:

```

```

-----

Library IEEE;

```

```

use IEEE.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;

```

```

library machxo3d;
use machxo3d.all;

```

```

entity mux2x8 is
    port(S: in STD_LOGIC;
          X0, X1: in STD_LOGIC_VECTOR(7 downto 0);
          F: out STD_LOGIC_VECTOR(7 downto 0));
end mux2x8;

```

```

architecture Behavioral of mux2x8 is
begin

```



```

        mux_behavior: process(X0, X1, S)
        begin
            if S = '0' then
                F <= X0;
            else
                F <= X1;
            end if;
        end process;
    end Behavioral;

```

```

Library IEEE;
use IEEE.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;

```

```

entity reg8 is
    port(load: in STD_LOGIC;
          D: in STD_LOGIC_VECTOR(7 downto 0);
          clk: in STD_LOGIC;
          Q: out STD_LOGIC_VECTOR(7 downto 0));
end reg8;

```

```

architecture BEHAVIOR of reg8 is
begin
    reg8_behavior: process
    begin
        wait until clk'event and clk = '1';
        if load = '1' then
            Q <= D;
        else
            null;
        end if;
    end process;
end BEHAVIOR;

```

```

Library IEEE;
use IEEE.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;

```

```

entity comparator is
    port(A, B: in STD_LOGIC_VECTOR(7 downto 0);
          AeqB, AgB: out STD_LOGIC);
end comparator;

```

architecture BEHAVIOR of comparator is

begin

 P1: process(A, B)

 begin

 if A > B then

 AgB <= '1';

 AeqB <= '0';

 elsif A = B then

 AgB <= '0';

 AeqB <= '1';

 else

 AgB <= '0';

 AeqB <= '0';

 end if;

 end process;

end BEHAVIOR;

Library IEEE;

use IEEE.std_logic_1164.all;

use ieee.std_logic_unsigned.all;

use ieee.numeric_std.all;

entity cond_inv is

 port(inv: in STD_LOGIC;

 Din: in STD_LOGIC_VECTOR(7 downto 0);

 Dout: out STD_LOGIC_VECTOR(7 downto 0));

end cond_inv;

architecture BEHAVIOR of cond_inv is

begin

 P1: process(INV, Din)

 begin

 if INV = '1' then

 Dout <= not(Din);

 else

 Dout <= Din;

 end if;

 end process;

end BEHAVIOR;

Library IEEE;

use IEEE.std_logic_1164.all;

use ieee.std_logic_unsigned.all;

```

use ieee.numeric_std.all;

entity RCA is
    port(X, Y: in STD_LOGIC_VECTOR(7 downto 0);
         Cin: in STD_LOGIC;
         S: out STD_LOGIC_VECTOR(7 downto 0));
end RCA;

architecture BEHAVIOR of RCA is
begin
    S <= (X + Y) + Cin;
end BEHAVIOR;

```

Appendix B (GCD Code):

```

-----
-- Company: University of Arkansas (RIOT)
-- Engineer: Tyler Cash
--
-- Create Date: 27Feb2026
-- Last Updated: 27Feb2026
-- Design Name: GCD
-- Module Name: GCD TOP
-- Project Name: GCD
-- Target Devices: LCMXO3D-9400HC-5BG256C (MachXO3D_BreakoutBrd)
-- Tool versions: Lattice Diamond_x64 Build 3.12.0.240.2
-- Description:
-- Top module for implementing a GCD.
--
---- PinOut:
-- Signal      CPLD_Pin Description
--Push Buttons
-- Clk          J2      Clk_In (Input) [Onboard btn[0]]
-- PB_1         J5      PB1   (Input) [Onboard btn[1]]
-- PB_2         H2      PB2   (Input) [Onboard btn[2]]
-- PB_3         J1      PB3   (Input) [Onboard btn[3]]
--
-- Switches
-- X(0)         V2      SW0   (Input) [Onboard sw[0]]
-- X(1)         U2      SW1   (Input) [Onboard sw[1]]
-- X(2)         U1      SW2   (Input) [Onboard sw[2]]
-- X(3)         T2      SW3   (Input) [Onboard sw[3]]
-- X(4)         T1      SW4   (Input) [Onboard sw[4]]

```

```

-- X(5)      R2    SW5      (Input) [Onboard sw[5]]
-- X(6)      R1    SW6      (Input) [Onboard sw[6]]
-- X(7)      P2    SW7      (Input) [Onboard sw[7]]
-- resetP1   SW8      (Input) [Onboard sw[8]]
-- Xdat      N2    SW9      (Input) [Onboard sw[9]]
-- YrqstV2   SW10 (Input) [Onboard sw[10]]
-- Sw_11     P1    SW11 (Input) [Onboard sw[11]]
-- SW_12     P1    SW12 (Input) [Onboard sw[12]]
-- SW_13     P1    SW13 (Input) [Onboard sw[13]]
-- SW_14     P1    SW14 (Input) [Onboard sw[14]]
-- SW_15     P1    SW15 (Input) [Onboard sw[15]]
--
-- LEDs
-- Y(0)      G1    LED0 (Output) [Onboard led[0]]
-- Y(1)      G2    LED1 (Output) [Onboard led[1]]
-- Y(2)      F1    LED2 (Output) [Onboard led[2]]
-- Y(3)      F2    LED3 (Output) [Onboard led[3]]
-- Y(4)      E1    LED4 (Output) [Onboard led[4]]
-- Y(5)      E2    LED5 (Output) [Onboard led[5]]
-- Y(6)      E3    LED6 (Output) [Onboard led[6]]
-- Y(7)      E5    LED7 (Output) [Onboard led[7]]
-- Ydat      E6    LED8 (Output) [Onboard led[8]]
-- XrqstC3   LED9 (Output) [Onboard led[9]]
-- LED_10    B2 LED10 (Output) [Onboard led[10]]
-- LED_11    A2 LED11 (Output) [Onboard led[11]]
-- LED_12    B3 LED12 (Output) [Onboard led[12]]
-- LED_13    A3 LED13 (Output) [Onboard led[13]]
-- LED_14    B4 LED14 (Output) [Onboard led[14]]
-- LED_15    A4 LED15 (Output) [Onboard led[15]]
--
-- Revisions:--
--
-- Revision 0.01 -
-- File Created;
--
--
-- Additional Comments:
--
--
-----

```

```

Library IEEE;
use IEEE.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

```

```

use ieee.numeric_std.all;

library machxo3d;
use machxo3d.all;

entity GCD_chip is
    port(A, B: in STD_LOGIC_VECTOR(7 downto 0);
          Xdat, Yrqst, reset, clk: in STD_LOGIC;
          Y: out STD_LOGIC_VECTOR(7 downto 0);
          Ydat, Xrqst: out std_logic);
end;

architecture BEHAVIOR of GCD_chip is
    type state_type is (S0, S1);
    signal CS, NS: state_type;
    signal Ai, Bi, Ao, Bo, Aoi, Boi, Sum: STD_LOGIC_VECTOR(7 downto 0);
    signal S, LDa, LDb, AgB, AeqB, INVa, INVb: STD_LOGIC;

    component reg8
        port(load: in STD_LOGIC;
              D: in STD_LOGIC_VECTOR(7 downto 0);
              clk: in STD_LOGIC;
              Q: out STD_LOGIC_VECTOR(7 downto 0));
    end component;

    component mux2x8
        port(S: in STD_LOGIC;
              X0, X1: in STD_LOGIC_VECTOR(7 downto 0);
              F: out STD_LOGIC_VECTOR(7 downto 0));
    end component;

    component cond_inv
        port(inv: in STD_LOGIC;
              Din: in STD_LOGIC_VECTOR(7 downto 0);
              Dout: out STD_LOGIC_VECTOR(7 downto 0));
    end component;

    component comparator
        port(A, B: in STD_LOGIC_VECTOR(7 downto 0);
              AeqB, AgB: out STD_LOGIC);
    end component;

    component RCA is
        port(X, Y: in STD_LOGIC_VECTOR(7 downto 0);

```

```

        Cin: in STD_LOGIC;
        S: out STD_LOGIC_VECTOR(7 downto 0));
end component;

```

```
begin
```

```

    Amux: mux2x8 -- Inputs: Selector Switch, Sum, A Input Output: Ai (to feed into the
Register)

```

```

        port map(
            S => S,
            X0 => Sum,
            X1 => A,
            F => Ai
        );

```

```

    Bmux: mux2x8 -- Inputs S, LDb, Bi Outputs: Bo

```

```

        port map(
            S => S,
            X0 => Sum,
            X1 => B,
            F => Bi
        );

```

```

    Areg: reg8

```

```

        port map(
            load => LDa,
            D => Ai,
            clk => clk,
            Q => Ao
        );

```

```

    Breg: reg8

```

```

        port map(
            load => LDb,
            D => Bi,
            clk => clk,
            Q => Bo
        );

```

```

    comp: comparator

```

```

        port map(
            A => Ao,
            B => Bo,
            AeqB => AeqB,
            AgB => AgB
        );

```

```

    );

ciA: cond_inv
    port map(
        inv => INVa,
        Din => Ao,
        Dout => Aoi
    );

ciB: cond_inv
    port map(
        inv => INVb,
        Din => Bo,
        Dout => Boi);

adder: RCA
    port map(
        X => Aoi,
        Y => Boi,
        Cin => '1',
        S => Sum);

-- State Logic for ASM
comb: process(CS, Xdat, AeqB, AgB, Yrqst) -- Sensitivity List
begin
    -- Set Defaults
    Xrqst <= '0';
    LDa <= '0';
    LDb <= '0';
    S <= '0';
    INVa <= '0';
    INVb <= '0';
    Ydat <= '0';
    NS <= S0;

    case CS is

        --State S0
        when S0 =>
            -- ASsert Xrqst, LDa, LDb, and S0
            Xrqst <= '1';
            LDa <= '1';
            LDb <= '1';
            S <= '1';
    end case;
end process;

```

```

        --Check if input data is ready / valid
        if Xdat = '0' then
            NS <= S0;
        else
            NS <= S1;
        end if;

    --State S1
    when S1 =>
        --Check if A is equal to B3
        if AeqB = '0' then
            NS <= S1;
            if AgB = '0' then
                INVa <= '1'; -- Invert A
                LDdb <= '1'; -- Load B
            else
                INVb <= '1'; -- Invert B
                LDa <= '1'; -- Load A
            end if;
        else
            if Yrqst = '0' then
                NS <= S1;
            else
                Ydat <= '1';
                NS <= S0;
            end if;
        end if;
    end case;
end process;

-- Update Current State based on Next State Logic
sync: process
begin
    wait until clk'event and clk = '1';
    if reset = '0' then
        CS <= S0;
    else CS <= NS;
    end if;
    Y <= Ao;
end process;
end;

```


Appendix C (GCD Testbench):

```
-----
-- Company: University of Arkansas (RIOT)
-- Engineer: Tyler Cash
--
-- Create Date: 27Feb2026
-- Last Updated: 27Feb2026
-- Design Name: GCD
-- Module Name: GCD TOP
-- Project Name: GCD
-- Target Devices: LCMXO3D-9400HC-5BG256C (MachXO3D_BreakoutBrd)
-- Tool versions: Lattice Diamond_x64 Build 3.12.0.240.2
-- Description:
-- Top module for implementing a GCD.
--
---- PinOut:
-- Signal      CPLD_Pin Description
--Push Buttons
-- Clk          J2      Clk_In (Input) [Onboard btn[0]]
-- PB_1         J5      PB1   (Input) [Onboard btn[1]]
-- PB_2         H2      PB2       (Input) [Onboard btn[2]]
-- PB_3         J1      PB3       (Input) [Onboard btn[3]]
--
-- Switches
-- X(0)         V2      SW0       (Input) [Onboard sw[0]]
-- X(1)         U2      SW1       (Input) [Onboard sw[1]]
-- X(2)         U1      SW2       (Input) [Onboard sw[2]]
-- X(3)         T2      SW3       (Input) [Onboard sw[3]]
-- X(4)         T1      SW4       (Input) [Onboard sw[4]]
-- X(5)         R2      SW5       (Input) [Onboard sw[5]]
-- X(6)         R1      SW6       (Input) [Onboard sw[6]]
-- X(7)         P2      SW7       (Input) [Onboard sw[7]]
-- resetP1      SW8           (Input) [Onboard sw[8]]
-- Xdat         N2      SW9       (Input) [Onboard sw[9]]
-- YrqstV2      SW10      (Input) [Onboard sw[10]]
-- Sw_11        P1      SW11      (Input) [Onboard sw[11]]
-- SW_12        P1      SW12      (Input) [Onboard sw[12]]
-- SW_13        P1      SW13      (Input) [Onboard sw[13]]
-- SW_14        P1      SW14      (Input) [Onboard sw[14]]
-- SW_15        P1      SW15      (Input) [Onboard sw[15]]
--
-- LEDs
-- Y(0)         G1      LED0      (Output) [Onboard led[0]]
```

```

-- Y(1)      G2    LED1  (Output) [Onboard led[1]]
-- Y(2)      F1    LED2  (Output) [Onboard led[2]]
-- Y(3)      F2    LED3  (Output) [Onboard led[3]]
-- Y(4)      E1    LED4  (Output) [Onboard led[4]]
-- Y(5)      E2    LED5  (Output) [Onboard led[5]]
-- Y(6)      E3    LED6  (Output) [Onboard led[6]]
-- Y(7)      E5    LED7  (Output) [Onboard led[7]]
-- Ydat      E6    LED8  (Output) [Onboard led[8]]
-- XrqstC3   LED9  (Output) [Onboard led[9]]
-- LED_10    B2    LED10  (Output) [Onboard led[10]]
-- LED_11    A2    LED11  (Output) [Onboard led[11]]
-- LED_12    B3    LED12  (Output) [Onboard led[12]]
-- LED_13    A3    LED13  (Output) [Onboard led[13]]
-- LED_14    B4    LED14  (Output) [Onboard led[14]]
-- LED_15    A4    LED15  (Output) [Onboard led[15]]

```

```

--
-- Revisions:--
--
-- Revision 0.01 -
-- File Created;
--
--
-- Additional Comments:
--
--

```

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

```

```

ENTITY GCD_TB IS
End GCD_TB;

```

ARCHITECTURE behavior OF GCD_TB IS

```

    COMPONENT GCD_chip
    Port (
        A : IN std_logic_vector(7 downto 0);
        B : IN std_logic_vector(7 downto 0);
        Xdat : IN std_logic;
        Yrqst : IN std_logic;

```

```

        reset : IN std_logic;
        clk : IN std_logic;
        Y : OUT std_logic_vector(7 downto 0);
        Ydat : OUT std_logic;
        Xrqst : OUT std_logic
    );
END COMPONENT;

SIGNAL A_Test : std_logic_vector(7 downto 0);
SIGNAL B : std_logic_vector(7 downto 0);
SIGNAL Xdat : std_logic;
SIGNAL Yrqst : std_logic;
SIGNAL reset : std_logic;
SIGNAL clk : std_logic;
SIGNAL Y : std_logic_vector(7 downto 0);
SIGNAL Ydat : std_logic;
SIGNAL Xrqst : std_logic;

--constant read_Time: time :=8680 ns; --for 115,200 Baud
--constant read_Time: time :=104100 ns; --for 9,600 Baud

BEGIN

-- Please check and add your generic clause manually
    uut: GCD_chip PORT MAP (
        A => A_Test,
        B => B,
        Xdat => Xdat,
        Yrqst => Yrqst,
        reset => reset,
        clk => clk,
        Y => Y,
        Ydat => Ydat,
        Xrqst => Xrqst
    );

    -- Clock process
    CLOCK: process
    begin
        clk <='0';
        wait for 25 ns;
        clk <='1';
        wait for 25 ns;
    end process;

```

```
END PROCESS;
```

```
-- Stimulate process
```

```
INPUTS: process
```

```
begin
```

```
    wait for 100 ns;
```

```
    reset <= '0';
```

```
    wait for 100 ns;
```

```
    reset <= '1';
```

```
    Yrqst <= '0';
```

```
    Xdat <= '0';
```

```
    wait for 100 ns;
```

```
    A_Test <= "00011001"; -- A = 25
```

```
    B <= "00011110"; -- B = 30
```

```
    wait for 100 ns;
```

```
    Xdat <= '1';
```

```
    wait for 100 ns;
```

```
    Xdat <= '0';
```

```
    wait for 500 ns;
```

```
    A_Test <= "01100100";
```

```
    B <= "00001010";
```

```
    wait for 100 ns;
```

```
    Xdat <= '1';
```

```
    wait for 100 ns;
```

```
    Xdat <= '0';
```

```
    wait for 500 ns;
```

```
    Yrqst <= '1';
```

```
    wait for 50 ns;
```

```
    Xdat <= '1';
```

```
    Yrqst <= '0';
```

```
    A_Test <= "00001001";
```

```
    B <= "00001001";
```

```
    wait for 100 ns;
```

```
    Xdat <= '0';
```

```
        wait for 50 ns;

        Yrqst <= '1';
        wait for 50 ns;

        Yrqst <= '0';
        wait for 200 ns;

        Xdat <= '1';
        Yrqst <= '0';
        A_Test <= x"0F";
        B <= x"14";
        wait for 100 ns;

        Xdat <= '0';
        wait for 50 ns;

        Yrqst <= '1';
        wait for 200 ns;

        wait;
    end process;
end;
```